Abstract

The basis of the Google search engine is the Page Rank algorithm. It was created in 1996 while Larry Page and Sergey Brin were doctoral candidates at Stanford University. To rank websites in the search results and identify their relevance and importance, the algorithm employs a complex mathematical formula. The formula considers a variety of variables, including the quantity and caliber of links heading to a page and the content's significance. The Page Rank algorithm was added to the Google search engine, which swiftly overtook all competitors due to the precision and relevancy of its search results. This report provides a comprehensive overview of the Page Rank algorithm and the Google search engine, including the development, working, and Mathematical Components behind the algorithm and the search engine.

1. Introduction:

The Page Rank algorithm has become the cornerstone of the Google search engine. The Page Rank algorithm uses a sophisticated mathematical formula to determine the relevance and importance of web pages and to rank them in the search results. The goal of the algorithm is to provide users with the most relevant and high-quality search results possible, making it an essential tool for anyone using the Internet to search for information.

The Page Rank algorithm works by analyzing the structure of the World Wide Web and the relationships between web pages. The algorithm assigns a score, called the Page Rank score, to each web page based on the number of links pointing to that page and the quality of those links. The higher the Page Rank score, the more relevant and important a web page is considered to be. The Page Rank algorithm also takes into account other factors such as the relevance of the content on a page, the popularity of the site, and the number of pages on the site.

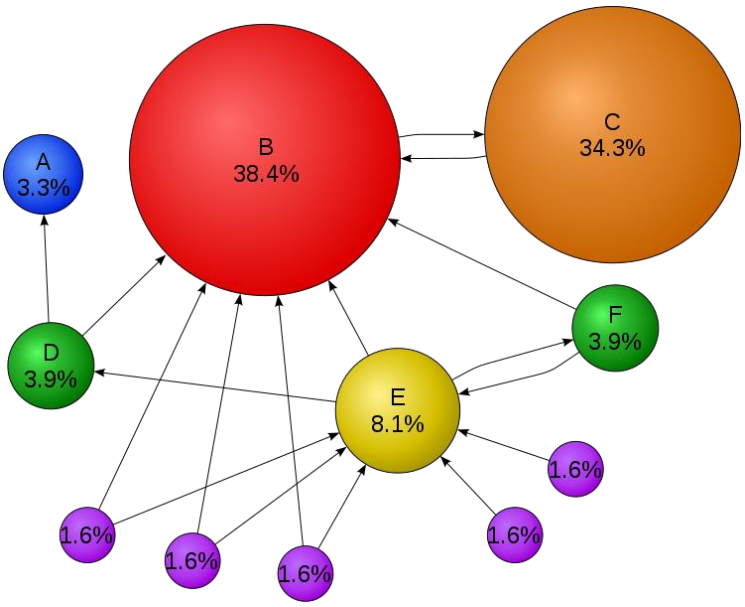

Figure 1.1 PageRank Algorithm - Web

When a user conducts a search on Google, the Page Rank algorithm is used to sort the search results by relevance and importance. The search results are then displayed to the user, with the most relevant and important results appearing at the top. This makes it easier for the user to find the information they are looking for, as the most relevant and high-quality results are presented first.

The Page Rank algorithm has had a profound impact on the field of search engine optimization (SEO). SEO is the process of optimizing a website

to rank higher in the search engine results pages (SERPs). By understanding the factors that influence a website's Page Rank score, website owners can improve their website's ranking in the search results, making it easier for users to find their site.

In Laymen's terms, The PageRank algorithm outputs a probability distribution used to represent the likelihood that a person randomly clicking on links will arrive at any particular page. PageRank can be calculated for collections of documents of any size.

In conclusion, the Page Rank algorithm is an essential component of the Google search engine, providing users with accurate and relevant search results. The algorithm's impact on the field of search engine optimization has also been significant, with website owners working to improve their website's Page Rank score to rank higher in the search results. This paper goes into further detail on and analyses the mathematics, working, and development aspects of it.

2. Methodology:

The PageRank algorithm uses a recursive formula to calculate the importance of each web page. The basic idea is that a web page is important if many other web pages link to it, and the importance of these linking pages is also taken into account. The algorithm works by assigning a score of 1 to each page and then repeatedly updating the scores based on the links between pages. The process continues until the scores no longer change, at which point a final score is assigned to each page [5].

The formula for calculating the PageRank of a web page is:

$$PR(A) = (1-d) + d\,(PR(T1)/C(T1) + \ldots + PR(Tn)/C(Tn))$$ [6]

Where,

PR(A) is the PageRank of page A

d is a damping factor, typically set to 0.85

PR(Ti) is the PageRank of pages linking to page A

C(Ti) is the number of outgoing links from page Ti

"PR(A) can be calculated using a simple iterative algorithm, and corresponds to the principal eigenvector of the normalized link matrix of the web." [6]

In another Form,

$$PR(P_i) = \frac{1-d}{N} + d \sum_{pj \in M(pi)} \frac{PR(Pj)}{L(Pj)}$$

Where,

$p_1, p_2, p_3, \ldots, p_n$ are the pages

$M(p_i)$ is the set of pages that links to p_i

$L(p_i)$ is the number of outgoing links on the page p_i

N is the total number of pages [1].

The damping factor, d, is used to reduce the influence of random jumps, or "teleporting," between pages. In other words, it is used to ensure that the algorithm does not get stuck in an endless loop of following links from one page to another. The damping factor is typically set to 0.85, which represents a trade-off between following links and reducing the influence of random jumps. The damping factor is important because there is no condition that webpages on the internet should be linked to one another.

Let us consider two sets of pages

A = {s1, s2, s3, s4, s5}

B = {s6, s7, s8}

Since there is no link between set A and set B, if a person starts to surf around from one page in a set, there is no possibility that the person will land on the pages in the other set. To avoid this condition, the damping factor is introduced. 85% of the time the random surfer surfaces the pages, while 15 % of the time the random surfer is switched to a completely random page on the internet [4].

The PageRank algorithm is designed to be used in a large-scale web search engine, where billions of pages must be processed and ranked. To handle this complexity, the algorithm is typically implemented as a distributed algorithm, where the work is divided among many computers. The value of the damping factor is chosen as 0.85 because the approximate PageRank converges quickly with limited iterations. For larger values, it requires more iterations and moreover numeric instability arises when the damping factor (d) is too close to 1 [3].

One of the key features of the PageRank algorithm is its ability to handle the structure of the web graph, which is the representation of the links between web pages. The web graph is a very complex structure, and the PageRank

algorithm is designed to handle this complexity by considering the structure of the links between pages.

For example, the PageRank algorithm takes into account the number of links pointing to a page, as well as the quality of those links. Pages that have many high-quality links pointing to them are likely to have a higher PageRank score, while pages with few or low-quality links will have a lower score. The quality of a link is determined by the PageRank of the page linking to it [10].

The PageRank algorithm also considers the structure of the links between pages. For example, pages that have many links pointing to them from high-quality pages are likely to have a higher PageRank score, while pages with few or low-quality links from other pages will have a lower score. This structure of the links between pages is referred to as the "web graph."

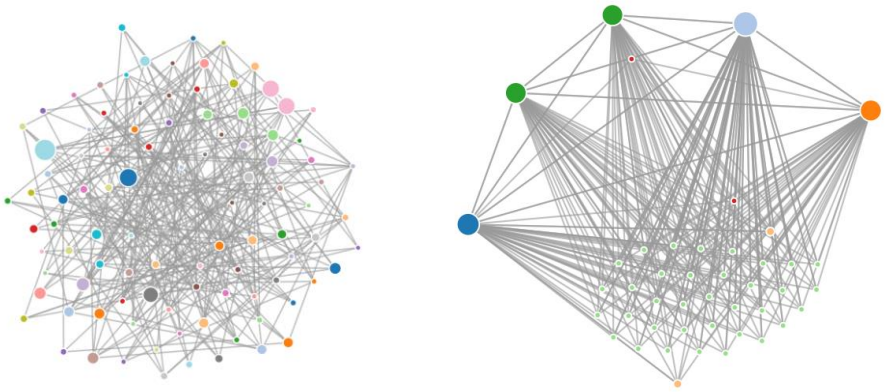

Figure 2.1 PageRank – Web Graph

Markov Chain is a mathematical model that describes a sequence of events where each event is dependent only on the previous event. In other words, it's a probabilistic model that defines a sequence of states where the probability of moving from one state to another depends only on the current state [8].

PageRank algorithm uses Markov Chain as a model to rank web pages based on their importance. The PageRank algorithm assumes that every web page on the internet can be represented as a node in a graph, and the links between the pages can be represented as edges. In this model, each page's importance is determined by the number and quality of links pointing to it. Pages with many high-quality links are considered more important and are therefore ranked higher.

In this model, web pages are treated as states in a Markov Chain. The probability of a user landing on a particular page depends on the probability of

landing on the previous page and the number of links from the previous page to the current page. The PageRank algorithm calculates the probability of a user being in a particular state (i.e., on a particular page), and the probabilities are used to rank the pages in the search results. In summary, PageRank uses Markov Chain as a model to represent web pages and their links, and the algorithm uses this model to calculate the importance of each page and rank them accordingly in the search results.

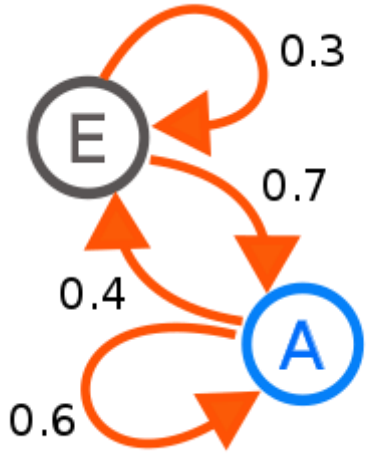

Figure 2.2 Markov Chain - Example

In addition to considering the structure of the links between pages, the PageRank algorithm also takes into account the content of the pages. Pages that contain relevant and valuable content are more likely to have a higher PageRank score, while pages with little or irrelevant content will have a lower score. This is why search engines like Google also use other algorithms, such as keyword analysis, to determine the relevance of a page to a particular search query.

The PageRank algorithm has been highly influential in the development of search engines and has had a significant impact on the way people find information on the web. However, it is not without its limitations.

Another limitation of the PageRank algorithm is its focus on popularity, which can sometimes lead to less relevant pages being ranked higher than more relevant pages. To address this issue, search engines like Google use additional algorithms, such as keyword analysis, to determine the relevance of a page to a particular search query.

In conclusion, the PageRank algorithm is a powerful tool for ranking web pages based on the number and quality of links pointing to them. While it has limitations, it remains an important component of search engines and has had a significant impact on the way people find information on the web.

Understanding the principles behind the PageRank algorithm is essential for anyone who wants to improve their website's search engine ranking and visibility.

3. Calculations:

The mathematical components of the PageRank algorithm can be represented as a Markov Chain, where each state represents a web page and the transition probability represents the likelihood of moving from one page to another based on the number and quality of links. The PageRank of a web page can be calculated as the steady-state probability distribution of the Markov Chain.

Let’s consider the following example,

Assume there are only 5 pages on the Internet (A, B, C, D, E),

- Page A links to E, B, D
- Page B links to C, D, E
- Page C doesn't link to any page
- Page D links to C
- Page E links to A, D

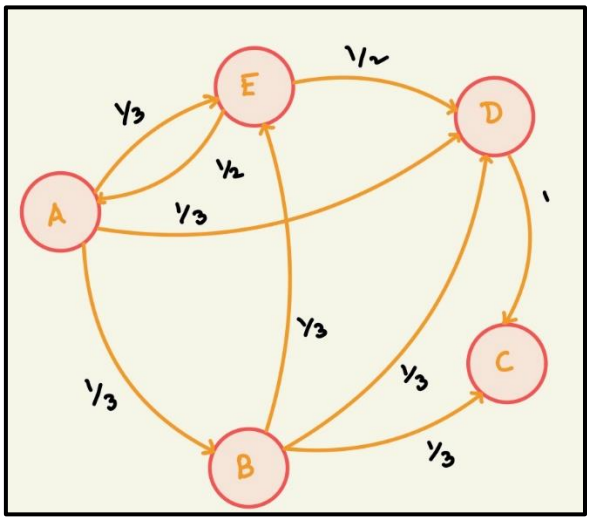

Figure 3.1 Example 1 – Page Rank Algorithm

Now representing the graph in terms of a matrix A based on the number of the number of outgoing matrix A.

Here each outgoing link is considered to have same probability as the all-other outgoing links in the page.

$$A=\begin{bmatrix} 0 & 0 & 0 & 0 & \frac{1}{2} \\ \frac{1}{3} & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{3} & 0 & 1 & 0 \\ \frac{1}{3} & \frac{1}{3} & 0 & 0 & \frac{1}{2} \\ \frac{1}{3} & \frac{1}{3} & 0 & 0 & 0 \end{bmatrix}$$

Also, the probability of going to a page is considered to have equal probabilities.

$$x=\begin{bmatrix} \frac{1}{5} \\ \frac{1}{5} \\ \frac{1}{5} \\ \frac{1}{5} \\ \frac{1}{5} \end{bmatrix}$$

The result of the next iterations is based on the previous. If the iterations continue for n times, the matrix converges to same value
Therefore,

$$x_{n+1}=A*x_n$$

This Equation is of the form,

$$\boxed{A.x= \lambda.x}$$

Importance vector x is an eigenvector of A with eigenvalue 1 [11].

Let P(0) = x,

P (1) = A * P (0)

.

.

.

P (n+1) = A * P (n)

In this example,

P (1) =

0.1150
0.0867
0.2567
0.2283
0.1433

P (20) =

0.0548
0.0455
0.1137
0.0833
0.0584

After the 12th iteration, the values have converged meaning P (12) ≅ P (20)
By changing the sum of P (20) to 1 since the probability should be 1,
We get,

P (20) =

0.1541
0.1280
0.3195
0.2341
0.1643

Now, the highest probability represents the page which will be ranked first, followed by the rest.
Rank of the pages are given as

Rank =

4

5

1

2

3

Here,

(1) Page C

(2) Page D

(3) Page E

(4) Page A

(5) Page B

4. Code:

4.1 MatLab:

4.1.1 PageRank.m

```
%% Page Rank Algorithm

% Without considering Damping Factor
% Page A links to E, B, D
% Page B links to C, D, E
% Page C doesn't link to any page
% Page D links to C
% Page E links to A, D

A1=[0 1/3 0 1/3 1/3;
    0 0 1/3 1/3 1/3;
    0 0 0 0 0;
    0 0 1 0 0;
    1/2 0 0 1/2 0];

A=A1';

% Considering Equal probability for landing in Pages
Initially
x=[1/5 1/5 1/5 1/5 1/5]';

% Concatnating Each iteration into a Matrix for our
understanding
k=[];
for i=1:20
    x=A*x;
    k=[k x];
end
k
x=x/sum(x);
rank1=rankMatrix(x);

% Considering Damping Factor
```

```
x1=[1/5 1/5 1/5 1/5 1/5]';

% Dampling Factor is set to 0.85
d=0.85;

% Concatnating Each iteration into
k1=[];

for i=1:20
    x1=d*(A*x1)+((1-d)/5);
    k1=[k1 x1];
end
k1
x1=x1/sum(x1);
rank2=rankMatrix(x1);
```

4.1.2 rankMatrix.m

```
function rankedMatrix = rankMatrix(matrix)
% Get the number of rows and columns in the matrix
[m, n] = size(matrix);
% Initialize a matrix to store the ranks
rankedMatrix = zeros(m, n);
% Loop through each column of the matrix
for j = 1:n
    % Get the current column of the matrix
    column = matrix(:, j);
    % Sort the current column in ascending order
    [sortedColumn, index] = sort(column);
    % Loop through the sorted column and assign ranks
    for i = 1:m
        rankedMatrix(index(i), j) = m-i+1;
    end
end
end
```

4.1.3 page_rank.m

```
function [x] = page_rank(G, damping, iterations)
% Here,
% G is the adjacency matrix of a graph
% alpha is the damping factor
```

```
% tol is the tolerance for the change in PageRank between
iterations
% pagerank is the final PageRank vector
% iterations is the number of iterations required to
converge

% Initialize PageRank vector
n = size(G, 1);
x = ones(n, 1) / n;
% Loop until change in PageRank is below the tolerance
for i = 1:iterations
    % Calculate new PageRank
    x = ((1 - damping) / n) + damping *(G*x);
    % Update change in PageRank
end
x = x/sum(x);
end
```

4.2 JavaScript:

4.2.1 PageRank.js

```
// Page Rank Algorithm

// Define the link matrix as a 2D array
var linkMatrix =
[
  [0, 1, 1, 0, 0],
  [1, 0, 1, 1, 0],
  [0, 0, 0, 1, 1],
  [1, 1, 1, 0, 1],
  [1, 1, 0, 1, 0]
];

// Define the number of pages
var numPages = linkMatrix.length;

// Define the damping factor
var dampingFactor = 0.85;

// Define the PageRank vector
var pageRank = [];
```

```
// Initialize the PageRank vector to a uniform
distribution
for (var i = 0; i < numPages; i++) {
  pageRank[i] = 1 / numPages;
}

// Iterate the PageRank algorithm until convergence
var convergence = false;
while (!convergence) {
  // Store the previous PageRank vector
  var previousPageRank = pageRank.slice();

  // Update the PageRank vector
  for (var i = 0; i < numPages; i++) {
    pageRank[i] = (1 - dampingFactor) / numPages;
    for (var j = 0; j < numPages; j++) {
      if (linkMatrix[j][i] === 1) {
        pageRank[i] += dampingFactor * previousPageRank[j]
/
          numLinks(linkMatrix[j]);
      }
    }
  }

  // Check for convergence
  convergence = isConverged(previousPageRank, pageRank);
}

// Print the PageRank vector
console.log("Page Rank Algorithm");
console.log("Matrix A : ");
console.log(linkMatrix);
console.log("Probabilities :");
console.log(pageRank);
// Printing the  Rank of the Pages using PageRank vector
console.log("Rank :")
console.log(rankArray(pageRank));
// Function to count the number of outgoing links from a
page
function numLinks(page) {
  var count = 0;
  for (var i = 0; i < page.length; i++) {
```

```
    count += page[i];
  }
  return count;
}

// Function to check for convergence of the PageRank
vector
function isConverged(previous, current) {
  var threshold = 0.0001;
  for (var i = 0; i < previous.length; i++) {
    if (Math.abs(previous[i] - current[i]) > threshold) {
      return false;
    }
  }
  return true;
}

function rankArray(arr) {
  let sortedArr = [...arr].sort((a, b) => b-a);
  let ranks = arr.map(val => sortedArr.indexOf(val) + 1);
  return ranks;
}
```

5. Conclusion:

A key element of how the internet works is the Page Rank algorithm. It is a technique created by the co-founders of Google, Larry Page and Sergey Brin, to assess the significance and relevance of a webpage based on the quantity and caliber of links going to it. The Page Rank algorithm, which still serves as the foundation of Google's search engine, has had a big impact on how data is found and displayed online. Because of its widespread impact, search engine optimization and website rankings are now synonymous with this term. The Page Rank algorithm, with its ongoing development and advancement, continues to be a crucial tool for navigating the internet's immense ocean of information.

References:

[1] PageRank, Wikipedia, https://en.wikipedia.org/wiki/PageRank#:~:text=PageRank%20is%20a%20link%20analysis,relative%20importance%20within%20the%20set.

[2] PageRank Algorithm Implementation, GeeksforGeeks, https://www.geeksforgeeks.org/page-rank-algorithm-implementation/

[3] PageRank as a Function of the Damping Factor, Computer & Information Science & Engineering, University of Florida, https://www.cise.ufl.edu/~adobra/DaMn/talks/damn05-santini.pdf

[4] How Google's PageRank Algorithm Works, YouTube, https://www.youtube.com/watch?v=J_q2XjWyyGI

[5] Google's PageRank Algorithm explained, Search Engine Watch, https://www.searchenginewatch.com/2018/10/25/googles-pagerank-algorithm-explained/

[6] The Anatomy of a Large-Scale Hypertextual Web Search Engine, Sergey Brin and Lawrence Page, Computer Science Department, Stanford University, http://infolab.stanford.edu/~backrub/google.html

[7] The Algorithm that started Google, YouTube, https://www.youtube.com/watch?v=qxEkY8OScYY

[8] Markov's Chain clearly explained, YouTube, https://www.youtube.com/playlist?list=PLM8wYQRetTxBkdvBtz-gw8b9lcVkdXQKV

[9] Markov Chain, Wikipedia, https://en.wikipedia.org/wiki/Markov_chain#:~:text=A%20Markov%20chain%20or%20Markov,attained%20in%20the%20previous%20event

[10] PageRank, CSCI E-80, Introduction to Artificial Intelligence with Python, https://cs50.harvard.edu/extension/ai/2020/fall/projects/2/pagerank/

[11] PageRank Algorithm, YouTube, https://www.youtube.com/watch?v=g_Ssc4NplFI

[12] PageRank Aguilar, Geneseo, https://www.geneseo.edu/~aguilar/public/assets/rw-2017/pagerank-notes-aguilar.pdf

[13] Use Page Rank Algorithm to rank websites, MathWorks, https://in.mathworks.com/help/matlab/math/use-page-rank-algorithm-to-rank-websites.html

[14] Page Rank Algorithm, Manchester University, https://personalpages.manchester.ac.uk/staff/yanghong.huang/teaching/MATH36032/html/labdemo16.html

[15] Google PageRank, University of Maryland, https://www.math.umd.edu/~immortal/MATH401/book/ch_google_pagerank.pdf

[16] PageRank Algorithm fully explained, Medium, https://towardsdatascience.com/pagerank-algorithm-fully-explained-dc794184b4af